Les pointeurs et l'allocation dynamique

- Les pointeurs
- L'allocation dynamique

Rado Rakotonarivo (Cours d'Asma Gabis)
rado.rakotonarivo@efrei.fr

P1 – 2024/2025

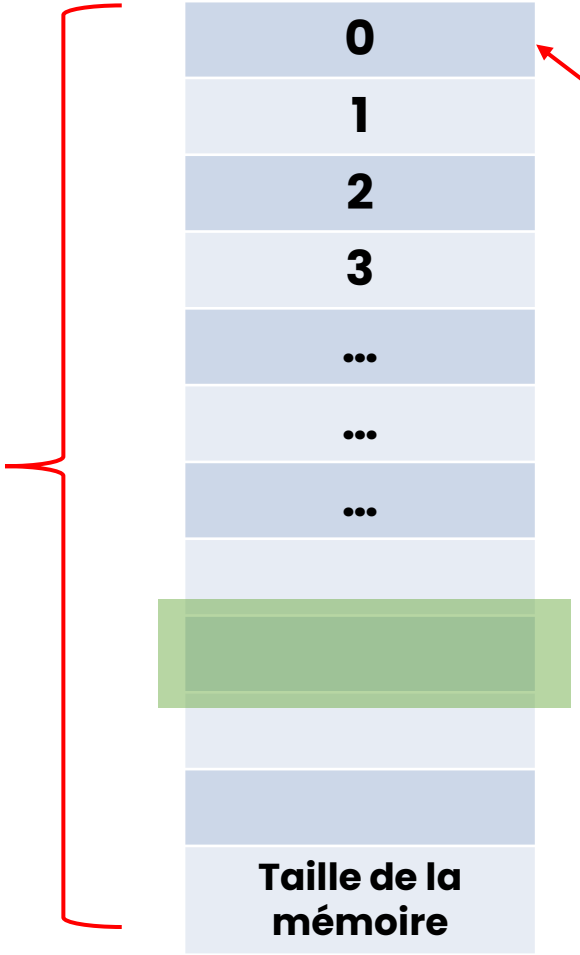
01

Les pointeurs

01

Les pointeurs

Représentation de la mémoire vive



Ressemble à un tableau divisé en cases numérotées séquentiellement .

Les numéros des cases mémoire commencent à 0.

La taille de chaque case est 1 octet (8 bits)

Les pointeurs

Stockage d'une variable en mémoire

- Lorsque une variable est déclarée, le système lui **alloue un emplacement mémoire** pour y stocker sa valeur.
- Cet emplacement aura sa propre adresse: **une adresse mémoire**.
- Les adresses mémoire sont représentées en hexadécimal.

```
char A ;
A = 10;
```

Adresses	Valeurs
… .	….
1599	
1600	10
1601	
1602	
1603	
… .	

A

Valeur stockée dans la variable A.

Adresse où est stockée la variable A

Nom symbolique de la variable

Les pointeurs

La taille des variables

- Les valeurs des variables **n'occupent pas toujours** une seule case en mémoire ➔ **Cela dépend de leurs types.**
- Le nombre d'octets qu'occupent les types de variable sont connus comme suit : **char** : 1 octet, **int** : 4 octets , **double** : 8 octets ➔ **Ce n'est pas toujours le cas :** Cela dépend des caractéristiques de la machine.
- Pour obtenir le nombre d'octets qu'occupe une variable d'un type donné, utiliser l'opérateur **sizeof** suivi du nom du type entre parenthèses.

```
#include <stdio.h>

int main()
{
    printf("Le type char occupe %d octets en mémoire. \n",sizeof(char));
    printf("Le type int occupe %d octets en mémoire. \n",sizeof(int));
    printf("Le type double occupe %d octets en mémoire. \n",sizeof(double));
    return 0;
}
```

```
Le type char occupe 1 octets en mémoire.
Le type int occupe 4 octets en mémoire.
Le type double occupe 8 octets en mémoire.
```

Les pointeurs

La taille des variables: Effet du stockage en mémoire

```
int main()
{
    int A = 3;
    return 0;
}
```

Adresses	Valeurs
… .	….
1599	
1600	3
1601	
1602	
1603	
… .	

4 octets

A

```
int main()
{
    double B = 3.8;
    return 0;
}
```

Adresses	Valeurs
….	….
1599	
1600	3.8
1601	
1602	
1603	
1604	
1605	
1606	
1607	
….	

8 octets

B

```
int main()
{
    char C = 'C';
    return 0;
}
```

Adresses	Valeurs
….	….
1599	
1600	67
1601	
1602	
1603	
….	

1 octet

C

Les pointeurs

Comment avoir l'adresse d'une variable ?

- L'opérateur **& (esperluette)** permet de donner l'adresse d'une variable.
- Deux formats peuvent être utilisés pour afficher l'adresse d'une variable :
  - **%u** : pour l'afficher comme étant de type : **Unsigned decimal integer**
  - **%X** : pour l'afficher comme étant de type : **Unsigned hexadecimal integer (majuscule)**

```c
int main()
{
    int var = 5;
    printf("la valeur de la variable var est : %d\n", var);

    printf("l'adresse de la variable var est : %u\n", &var);

    printf("l'adresse de la variable var en hexadecimal est : %X\n", &var);
    return 0;
}
```

```
la valeur de la variable var est : 5
l'adresse de la variable var est : 3911969208
l'adresse de la variable var en hexadecimal est : E92BE9B8
```

Les pointeurs

Comment avoir l'adresse d'une variable ?

- L'adresse affichée pour une variable dont le type occupe **plus d'1 octet** est l'adresse de **son premier octet**.
- La notation **&var** a déjà été vue dans la syntaxe de la fonction **scanf**.

stocker la valeur saisie au clavier dans la variable var en utilisant son adresse

```
int main()
{
    int var;

    scanf("%d", &var);
    printf("la valeur de la variable var est : %d\n", var);
    printf("l'adresse de la variable var en hexadecimal est : %X\n", &var);
    return 0;
}
```

Les pointeurs

Accès à une variable

Modes d'adressage

- Adressage direct
- **Adressage indirect**

Le mode utilisé jusque là: **L'accès au contenu de la variable se fait via son nom symbolique.**

Les pointeurs

Adressage indirect

- L'accès à une variable **A** donnée se fait au travers **d'une autre variable intermédiaire** **P**.

Comment ?

En stockant l'adresse de **A** dans la variable **P** ➔ **P** est donc une variable spéciale contenant une valeur qui visuellement ressemble à un entier mais qui représente **sémantiquement une adresse.**

Valeur de la variable P = adresse de la variable A

En mémoire

Adresse	valeur	
....		
1000	5424	P
1001		
1002		
...		
...		
5424	10	A
5425		

Adresse de la variable P

Adresse de la variable A

P pointe A

Les pointeurs

Qu'est ce qu'un pointeur ?

- Un pointeur est une variable "**spéciale**" qui est destinée à **contenir l'adresse d'une autre variable**.

Pourquoi utiliser un pointeur ?

- Un pointeur est utilisé pour l'accès à une variable lorsque :
  - **L'on ne veut pas** utiliser un accès direct à travers son nom symbolique
  - **L'on ne peut pas** accéder directement à cette variable

Avantages d'un pointeur

- Accès direct à la mémoire et manipulation de ses adresses ➔ un des principes fondamentaux les plus puissants de la programmation C (et C ++).
- Accès direct à la mémoire à partir d'un code source ➔ Les opérations liées à la mémoire sont très rapides.

Les pointeurs

Manipulation des pointeurs

Déclaration d'un pointeur

- Le caractère *** (astérisque)** permet de marquer le type pointeur.
- Une variable de type pointeur contiendra toujours une adresse, elle prend à la **déclaration** le **type de la variable qu'elle pointe**.
- Ainsi , un pointeur est déclaré comme suit:

Type* Nom_pointeur ; Ou **Type *Nom_pointeur ;**

Type de la donnée pointée

Nom symbolique donné au pointeur. Conventionnellement, ça commence par **p** ou se termine par **ptr**.

Signifie qu'il s'agit d'un type pointeur : peut être du côté du type ou du côté du nom de la variable

Les pointeurs

Manipulation des pointeurs

Déclaration d'un pointeur : exemples

```
int main()
{
    int A ; // déclaration d'une variable A
    char B; // déclaration d'une variable B
    int* Aptr = NULL ; // pointeur sur une variable de type int
    char *pB = NULL; // pointeur sur une variable de type char
    return 0;
}
```

Désigne le pointeur nul
(en général égal à zéro),
à utiliser pour initialiser
un pointeur

Les pointeurs

Manipulation des pointeurs

Opérateurs de référencement / déréférencement

- **L'opérateur « & »**
  - Appelé opérateur de référencement.
  - Lorsqu'il est appliqué sur **une variable** ➔ Il donne son **adresse**.
- **L'opérateur « * »**
  - Appelé opérateur de déréférencement.
  - Appliqué à une variable de type pointeur.
  - Il donne **la valeur** de **la variable pointée** par le pointeur (accès indirect à la variable).

Les pointeurs

Manipulation des pointeurs

Opérateurs de référencement / déréférencement

Analysons l'exemple ci-dessous :

```
#include <stdio.h>
int main()
{
    int A ; // déclaration d'une variable A
    int* Aptr = NULL; // // déclaration d'un pointeur sur une variable de type int
    A= 5; // initialisation de la variable A
    // initialisation d'un pointeur : pour pointer la variable A
    Aptr = &A; // affecter l'adresse de A à Aptr
    // Affichage de la valeur de A normalement
    printf("Accès direct : La valeur de A est: %d \n", A);
    // Affichage de l'adresse de A normalement
    printf("Accès direct : L'adresse de A est: %X \n", &A); // utilisation de l'&
    // Affichage de la valeur de A via le pointeur
    printf("Accès indirect : La valeur de A est: %d \n", *Aptr); // utilisation de l'*
    // Affichage de l'adresse de A via le pointeur  ==> afficher la valeur du pointeur
    printf("Accès indirect : L'adresse de A est: %X \n", Aptr);
    return 0;
}
```

```
Accès direct : La valeur de A est: 5
Accès direct : L'adresse de A est: E6ACF9B8
Accès indirect : La valeur de A est: 5
Accès indirect : L'adresse de de A est: E6ACF9B8
```

Les pointeurs

Manipulation des pointeurs

Attention aux confusions !!!!!

Soient les deux variables suivantes:

```
int c, *pc; // déclaration d'une variable et d'un pointeur sur une même ligne
```

`pc = c ;` Faux ! **pc** est un pointeur (adresse) alors que **c** ne l'est pas.

→ `pc = &c;` Correct ! **pc** est une adresse et **&c** l'est aussi.

`*pc = &c;` Faux ! ***pc** est une valeur alors que **&c** est une adresse.

→ `*pc = c;` Correct ! ***pc** est la valeur pointée par **pc** et c est une valeur de même type.

Les pointeurs

Pointeur de pointeur

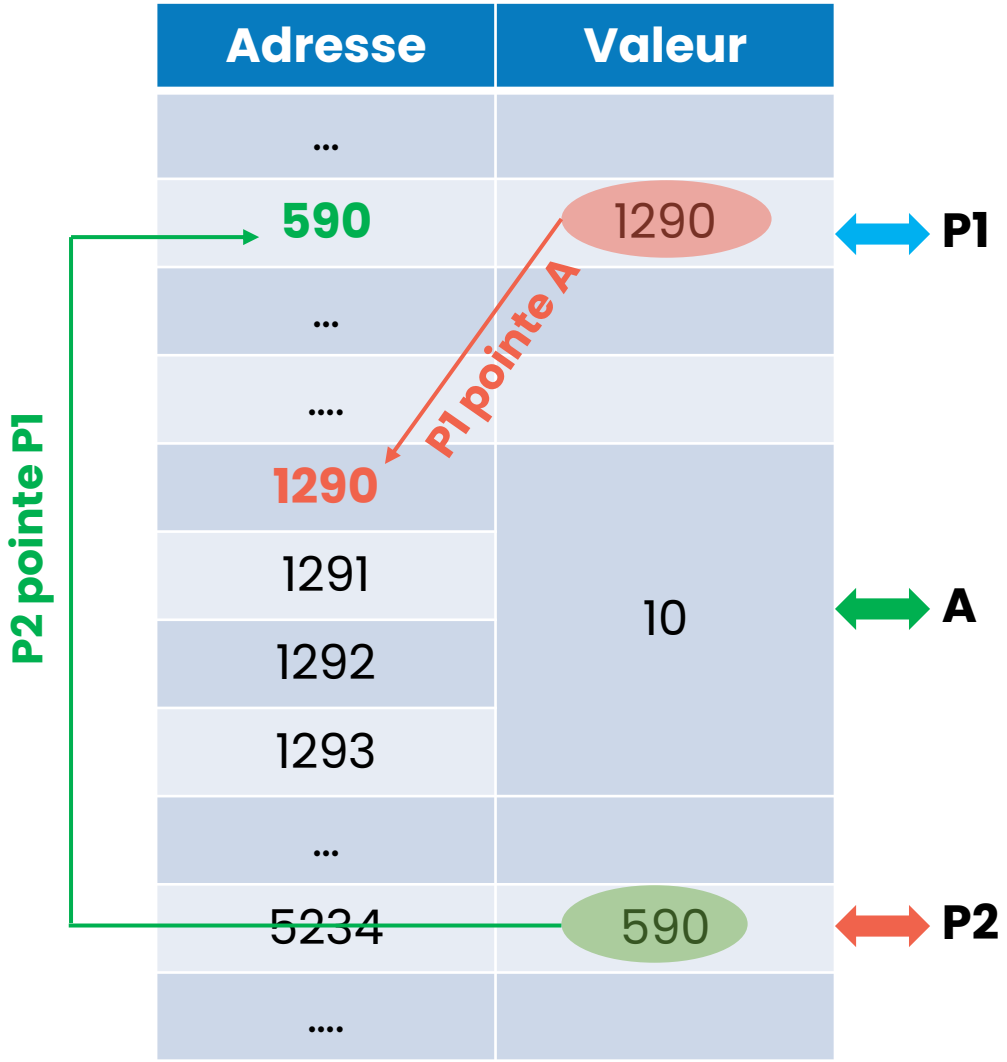
Que se passe t-il lorsqu'un pointeur **pointe un autre pointeur** et non pas une variable normale ? ➔ **Double pointeur**

Déclaration d'un double pointeur

Type **Nom_pointeur ;

Type de la donnée **finale** pointée

Signifie qu'il s'agit d'un **double** pointeur



Les pointeurs

Pointeur de pointeur

Exemple

```
int main()
{
    int num = 45 , *ptr , **ptr2ptr ;

    ptr = &num; // stocke l'adresse de la variable num

    ptr2ptr = &ptr; // stocke l'adresse du pointeur ptr

    return 0;
}
```

02

Pointeurs & Tableaux

02

Pointeurs & Tableaux

Quelle est la relation entre un tableau et un pointeur ?

- Le nom d'un tableau est lui-même une adresse et donc un pointeur
- C'est un pointeur constant sur le premier élément du tableau.
  - `double T[50];` ici **T** est un pointeur sur **T[0]** (i.e. **T=&T[0]**), qui est l'adresse du premier élément du tableau **T**.

Accès aux éléments d'un tableau

Rappel : Un tableau est stocké en mémoire sur une zone contigüe ➔ Les cases sont positionnées les unes après les autres:

```
int T[] = {12, 78, 2, 65, 90, 1, 11, 45, 81};
```

Cases	T[0]	T[1]	T[2]	T[3]	T[4]	T[5]	T[6]	T[7]	T[8]
Valeur	12	78	2	65	90	1	11	45	81

Représentation d'un tableau par les pointeurs

Une case **T[i]** occupe en mémoire le nombre d'octets du type de sa donnée. Ici : sizeof(int) = 4 octets

En mémoire :

Soit le tableau **T** suivant : `int T[] = {12, 78, 2, 65, 90, 1, 11, 45, 81};`

Cases	T[0]	T[1]	T[2]	T[3]	T[4]	T[5]	T[6]	T[7]	T[8]
Valeur	12	78	2	65	90	1	11	45	81
Adresse	**adr0**	**adr1**			**adr4**				

Ici adr0 = adresse du premier octet de la case T[0]

Ici adr1 = adresse du premier octet de la case T[1] = **adr0 + nombre d'octets occupés par T[0]**

Ici adr4 = adresse du premier octet de la case T[4] = **adr0 + nombre d'octets occupés par T[0] + ceux de T[1] + ceux de T[2] + ceux de T[3]**

Les données sont toutes de même type ➔ les cases ont toutes la même taille ➔ **adr4 = adr0 + (4 * sizeof(int))**

C'est pourquoi le nom du tableau + sa taille logique sont suffisants pour accéder à tous les éléments du tableau.

Représentation d'un tableau par les pointeurs

En programmation C :

Le nom d'un tableau est un pointeur, l'on peut donc écrire :

```
double *p;
double T[10];
p = T;
```

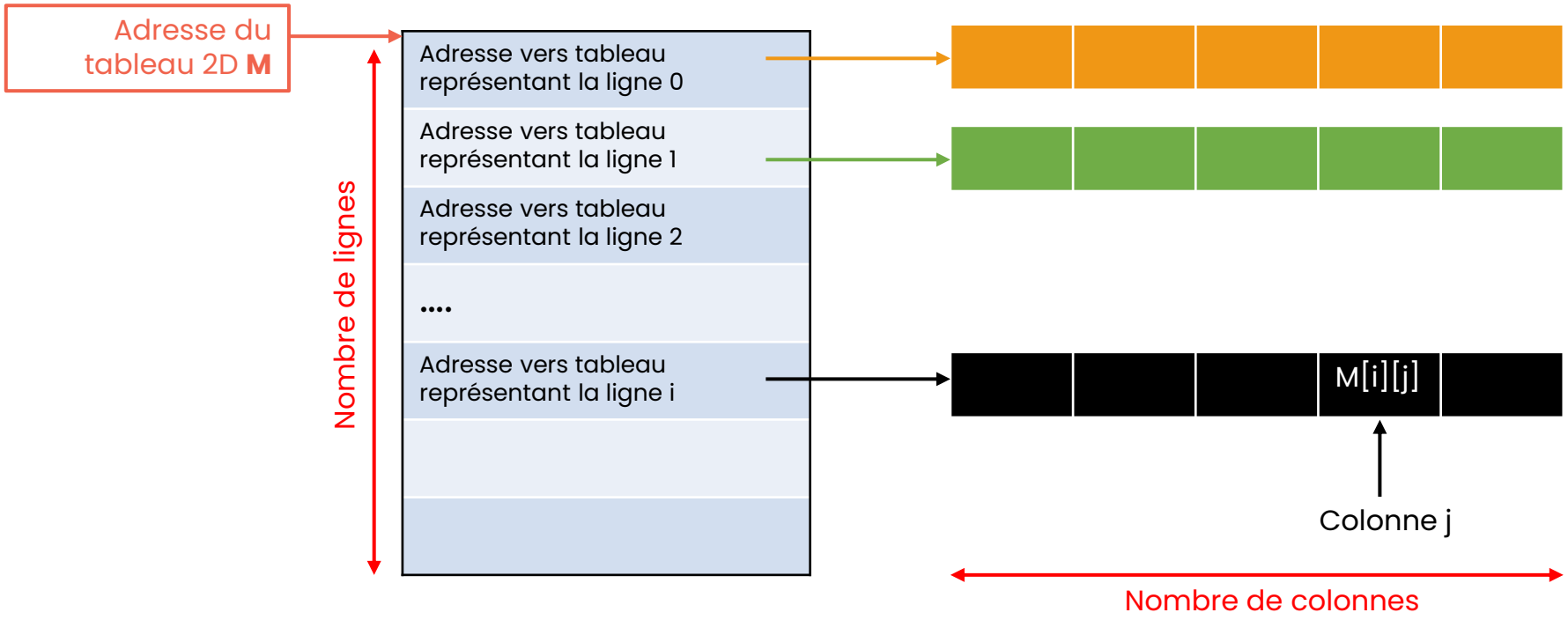
Ainsi, pour manipuler un tableau avec les pointeurs, les équivalences suivantes sont valides :

- *(T + i) ⇔ T[i] (i est multiplié implicitement par **sizeof(type des données de T)**)
- T+ i ⇔ &T[i]
- scanf("%d", T + i); ⇔ scanf("%d", &T[i]);
- printf("%d", *(T + i)); ⇔ printf("%d", T[i]);

Représentation d'un tableau 2D par les pointeurs

En mémoire :

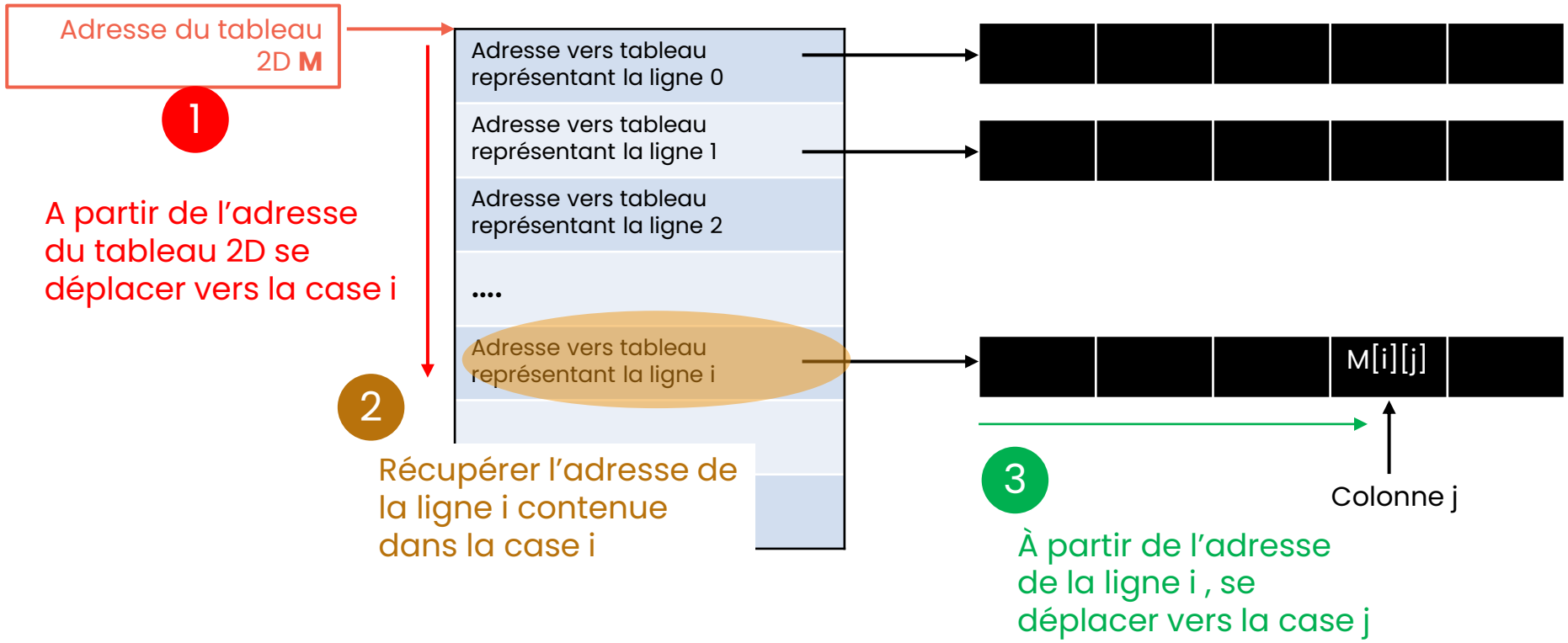
En mémoire, un tableau à 2D est représenté comme étant un tableau de tableaux



Représentation d'un tableau 2D par les pointeurs

En mémoire :

Pour accéder à une case d''un tableau 2D:



Représentation d'un tableau 2D par les pointeurs

En programmation C :

Soit le tableau 2D M suivant : `int M[3][2] ;`

Ainsi, pour le manipuler avec les pointeurs, les équivalences suivantes sont valides:

- *((*(M+i)) + j) ⇔ M[i][j] (i et j sont multipliés implicitement par **sizeof(type des données de M)**)
- (*(M+i)) + j ⇔ &M[i][j]
- scanf("%d", (*(M+i)) + j); ⇔ scanf("%d", &M[i][j]);
- printf("%d", *((*(M+i)) + j)); ⇔ printf("%d", M[i][j]);

03

L’allocation dynamique des tableaux

03

L'allocation dynamique des tableaux

Allocation de l'espace mémoire

Jusqu'à maintenant, la déclaration d'une variable entraîne automatiquement la réservation de l'espace mémoire nécessaire (**réservation statique**)

- Le nombre d'octets nécessaires est connu au temps de compilation;
- Le compilateur calcule cette valeur à partir du type de données de la variable.

MAIS

Souvent, nous devons travailler avec des données dont **nous ne pouvons prévoir le nombre et la grandeur lors de l'écriture du programme** :

- La taille des données est connue au temps d'exécution seulement.
- Nous devons éviter le gaspillage qui consiste à réserver l'espace maximal prévisible.

L'allocation dynamique des tableaux

Gestion de la mémoire

Zones mémoire

Pile (Stack)

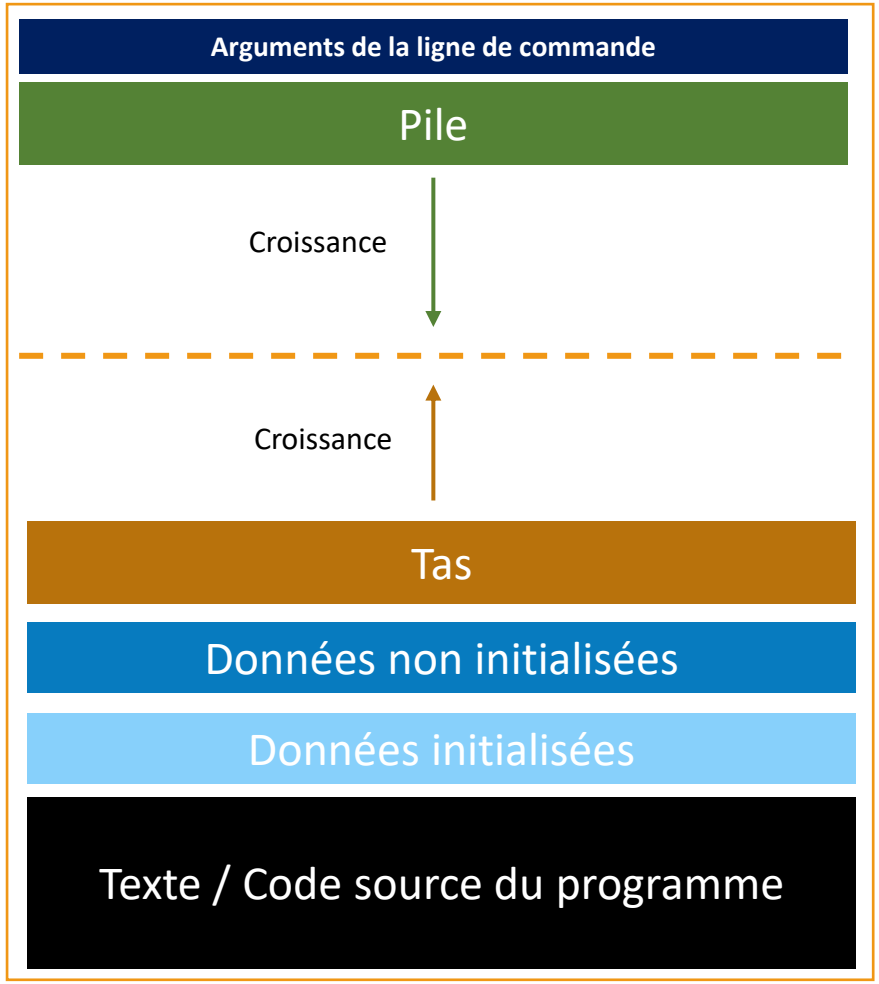
Tas (Heap)

- Utilisée pour les variables définies initialisées / non initialisées, variables locales
- Allocation automatique de la mémoire
- Libération automatique de la mémoire
- Traitement des données lors de la compilation du programme
- Accès rapide

- Utilisée pour l'allocation dynamique de la mémoire
- Allocation manuelle de la mémoire
- Libération manuelle de la mémoire
- Grande flexibilité dans la gestion de la mémoire
- Risque d'utilisation non correcte de la mémoire ➔ Nécessite plus de responsabilité de la part du programmeur

L'allocation dynamique des tableaux

Gestion de la mémoire



L'allocation dynamique des tableaux

Allocation de l'espace mémoire

Objectif :

Arriver à réserver ou de libérer de l'espace mémoire au fur et à mesure que nous en avons besoin pendant l'exécution du programme.

Solution

Allocation dynamique de la mémoire

En C: Avec la fonction malloc et l'opérateur sizeof

L'allocation dynamique des tableaux

Définition

- L'allocation dynamique de la mémoire permet de gérer "**manuellement**" la mémoire utilisée par votre programme.

- Utilise des fonctions de la bibliothèque **stdlib.h** pour l'allocation dynamique de la mémoire
  - **malloc()**: Alloue un nombre d'octets contigus et retourne un pointeur sur le premier octet.
  - **calloc():** Alloue l'espace pour un tableau en initialisant ses valeurs à zéro, et retourne un pointeur sur le premier élément.
  - **free()**: Libère l'espace alloué précédemment.
  - **realloc()**: Change la taille de l'espace alloué précédemment

L'allocation dynamique des tableaux

La fonction malloc()

- Pour "memory allocation »
- La fonction malloc() réserve un bloc mémoire d'une taille donnée et retourne un pointeur de type **void** (sans type) qui peut être "**casté**" en un pointeur du type souhaité.

Si l'espace sur votre machine est i**nsuffisant**, l'allocation échoue et ça retourne **NULL** (pointeur nul).

type* ptr = **(type*)** **malloc**(**taille**)

Type de la donnée pointée par ptr

Pour caster le type du pointeur retourné par la fonction malloc

Le nombre d'octets à réserver

L'allocation dynamique des tableaux

La fonction malloc()

- Généralisation pour le cas d'un tableau **T** de taille **N**.

Nom du tableau: choisi par l'utilisateur

Opérateur donnant le nombre d'octets qu'occupe un type en mémoire

```
type* T = (type*) malloc(N * sizeof(type));
```

Type de la donnée pointée par ptr

Pour caster le type du pointeur retourné par la fonction malloc

Nombre de cases souhaitées dans le tableau.

Type des données à stocker dans le tableau

L'allocation dynamique des tableaux

La fonction malloc()

- **Exemple 1**

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    // déclaration d'un pointeur vers un int
    int* A = NULL;

    // Allocation d'un bloc mémoire de 4 octets
    A = (int*) malloc(4);

    printf("Saisir un entier : \n");
    scanf("%d", A);
    printf("Valeur saisie = %d \n", *A);

    return 0;
}
```

```
Saisir un entier :
13
Valeur saisie = 13
```

L'allocation dynamique des tableaux

La fonction malloc()

- **Exemple 2**

```
Saisir le nombre de valeurs :
4
Saisir un entier :
1
Saisir un entier :
2
Saisir un entier :
3
Saisir un entier :
4
Les valeurs du tableau sont:
1        2        3        4
```

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    // déclaration d'un pointeur vers un int
    int* T = NULL;
    int taille;

    printf("Saisir le nombre de valeurs : \n");
    scanf("%d", &taille);

    // allocation d'un bloc mémoire de taille * 4 octets
    // équivalent à un tableau à "taille" cases
    T = (int*) malloc(taille * sizeof(int));

    // remplir le tableau
    for (int i = 0; i < taille; i++)
    {
        printf("Saisir un entier : \n");
        scanf("%d", &T[i]);
    }

    // affichage des éléments d'un tableau
    printf("Les valeurs du tableau sont: \n");
    for (int i = 0; i < taille; i++)
    {
        printf("%d\t\t", *(T+i));
    }
    return 0;
}
```

L'allocation dynamique des tableaux

La fonction calloc()

- Pour "contiguous allocation".
- La différence entre malloc() et calloc() est que:
  - malloc() alloue un seul bloc mémoire alors que **calloc() alloue des blocs multiples** tous de **même taille** et **initialisés à zéro**.

Nom du tableau: choisi par l'utilisateur

Taille du type en octets

```
type* T = (type*) calloc(N , sizeof(type));
```

Type de la donnée pointée par ptr

Pour caster le type du pointeur retourné par la fonction calloc

Nombre d'éléments souhaitées dans le tableau.

L'allocation dynamique des tableaux

La fonction calloc()

Exemple

```
Saisir le nombre de valeurs :
3
Saisir un entier :
1
Saisir un entier :
2
Saisir un entier :
3
Les valeurs du tableau sont:
1       2       3
```

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    // déclaration d'un pointeur vers un int
    int* T = NULL;
    int taille;

    printf("Saisir le nombre de valeurs : \n");
    scanf("%d", &taille);

    // allocation d'un bloc mémoire de taille * 4 octets
    // équivalent à un tableau à "taille" cases
    T = (int*) calloc(taille, sizeof(int));

    // remplir le tableau
    for (int i = 0; i < taille; i++)
    {
        printf("Saisir un entier : \n");
        scanf("%d", &T[i]);
    }

    // affichage des éléments d'un tableau
    printf("Les valeurs du tableau sont: \n");
    for (int i = 0; i < taille; i++)
    {
        printf("%d\t\t", *(T+i));
    }
    return 0;
}
```

L'allocation dynamique des tableaux

La fonction free()

L'espace mémoire alloué par **calloc()** ou **malloc()** n'est pas automatiquement libéré ➔ Il est nécessaire d'utiliser la fonction **free()** pour le libérer.

```
free(T);
```

Pointeur alloué dynamiquement

free() libère l'espace mais ne change pas la valeur du pointeur =

- Penser à **lui affecter NULL** après l'appel de free()
- Ne pas utiliser free() pour libérer un tableau statique

L'allocation dynamique des tableaux

La fonction realloc()

Si l'espace préalablement alloué s'avère insuffisant ou trop grand, vous pouvez changer sa taille en utilisant **realloc()**.

```
ptr = realloc (ptr, newsize);
```

Mise à jour du pointeur pour pointer sur le bloc pour qui la taille a été modifiée.

Pointeur alloué dynamiquement

Nouvelle taille souhaitée

S'il n'y a pas assez d'espace à la suite du bloc courant pour l'étendre, un nouveau bloc de la taille demandée est alloué, puis l'ancien bloc est copié dans le nouveau d'où la mise à jour de ptr.

L'allocation dynamique des tableaux

La fonction realloc()

Exemple

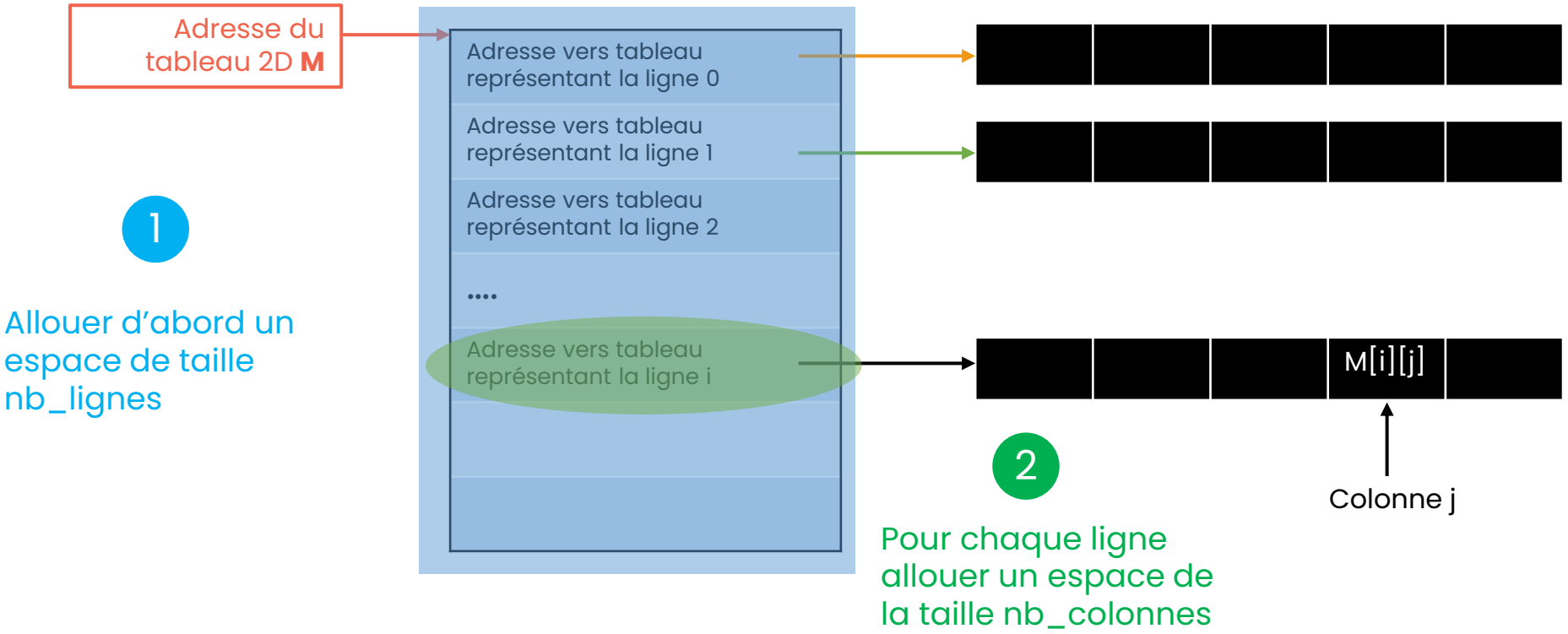
```
Enter size of array: 3
Address of previously allocated memory:
903889024   903889028   903889032
Enter new size of array: 5
903889024   903889028   903889032   903889036   903889040
```

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *ptr, i , n1, n2;
    printf("Enter size of array: ");
    scanf("%d", &n1);

    // allocation de n1 cases
    ptr = (int*) malloc(n1 * sizeof(int));

    printf("Address of previously allocated memory: \n");
    for(i = 0; i < n1; ++i)
        printf("%u\t",ptr + i);

    printf("\nEnter new size of array: ");
    scanf("%d", &n2);
    // allocation de n2 cases
    ptr = realloc(ptr, n2);
    for(i = 0; i < n2; ++i)
        printf("%u\t", ptr + i);

    free(ptr);
    ptr = NULL;
    return 0;
}
```

L'allocation dynamique des tableaux

Allocation dynamique d'un tableau 2D

Étapes



L'allocation dynamique des tableaux

Allocation dynamique d'un tableau 2D

Programme C

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int ** M;
    int nb_lig = 3;
    int nb_col = 4;

    // Allouer un espace de taille nb_lig
    // Chaque case est de type pointeur

    M = (int**) malloc(nb_lig * sizeof(int*));

    // Pour chaque ligne, allouer le nombre de colonnes nécessaire
    // ==> utiliser une boucle

    for (int i = 0; i < nb_lig; i++)
    {
        M[i] = malloc(nb_col * sizeof (int));
    }
    return 0;
}
```

L'allocation dynamique des tableaux

Application de free() sur un tableau 2D

Pour libérer un espace alloué dynamiquement pour un tableau à 2D, il faut :

1. D'abord libérer l'espace des petits tableaux (représentant les lignes)
2. Puis, libérer l'espace du tableau principal contenant leurs adresses.
3. Remettre le pointeur à NULL

Inverse de l'allocation dynamique

```
// Libérer l'espace des petits tableaux
for(int i = 0 ; i < nb_lig ; i++)
    free(M[i]);

// Libérer l'espace du tableau principal
free(M);

// Remettre le pointeur à NULL
M = NULL;
```

04

Application

04

Application

- **Exercice**

Quelle est la sortie du programme C suivant?

```
include <stdio.h>

int main(){
    /* an array with 5 elements */
    double balance[5] = {1000.0, 2.0, 3.4, 17.0, 50.0};
    double *p;
    int i;
    p = balance;

    /* output each array element's value */
    printf( "Array values using pointer\n");

    for ( i = 0; i < 5; i++ ) {
        printf("*(p + %d) : %f\n", i, *(p + i) );
    }

    printf( "Array values using balance as address\n");

    for ( i = 0; i < 5; i++ ) {
        printf("*(balance + %d) : %f\n", i, *(balance + i) );
    }
    return 0;
}
```

Application

- **Exercice : Solution**

Quelle est la sortie du programme C suivant?

```c
include <stdio.h>

int main(){
    /* an array with 5 elements */
    double balance[5] = {1000.0, 2.0, 3.4, 17.0, 50.0};
    double *p;
    int i;
    p = balance;

    /* output each array element's value */
    printf( "Array values using pointer\n");

    for ( i = 0; i < 5; i++ ) {
        printf("*(p + %d) : %f\n", i, *(p + i) );
    }

    printf( "Array values using balance as address\n");

    for ( i = 0; i < 5; i++ ) {
        printf("*(balance + %d) : %f\n", i, *(balance + i) );
    }
    return 0;
}
```

```
Array values using pointer
*(p + 0) : 1000.000000
*(p + 1) : 2.000000
*(p + 2) : 3.400000
*(p + 3) : 17.000000
*(p + 4) : 50.000000
Array values using balance as address
*(balance + 0) : 1000.000000
*(balance + 1) : 2.000000
*(balance + 2) : 3.400000
*(balance + 3) : 17.000000
*(balance + 4) : 50.000000
```